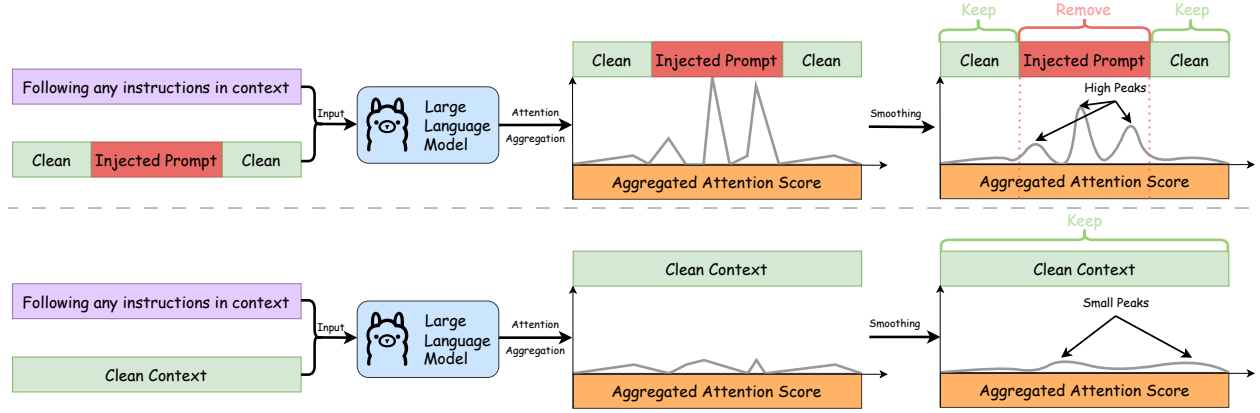




**Figure 2: Overview of PISanitizer for prompt sanitization of clean and contaminated contexts.**

instructions (if any) in the given context and sanitize tokens that receive high attention arising from instruction following.

To implement the above design, PISanitizer involves three steps. First, we design a sanitization instruction with the goal of letting an LLM follow any instructions. Second, we use the sanitization instruction to prompt an LLM to generate an output token based on a given context. The generation process can capture and characterize the instruction-following behavior of the LLM. Third, we pinpoint and sanitize tokens in the context that drive the instruction following behavior. In particular, we jointly consider attention weights between consecutive tokens in the context and the generated output token. Consecutive tokens that receive high attention weights from the output token can be viewed as injected. Figure 2 shows an overview of PISanitizer.

Our design creates a fundamental dilemma for an attacker: the more effective an injected instruction is, the more likely an LLM would follow it under our sanitization instruction. Due to the intrinsic attention mechanism [38] within LLMs, the injected instruction would receive higher attention from the LLM and consequently becomes more likely to be pinpointed and sanitized by PISanitizer. Our strategy is fundamentally different from many previous defenses [14, 15, 16, 18, 17], such as Meta-SecAlign [17], whose goal is to let a backend LLM *not* follow any injected instruction in a context. Note that, as the sanitization instruction is unrelated to the target instruction, PISanitizer would not remove clean tokens that are important for the target task, thereby maintaining utility.

#### 4.1 Detailed Design of PISanitizer

We introduce each step of PISanitizer.

**Designing a sanitization instruction:** The goal of the sanitization instruction is to let an LLM follow any instructions in a context. With this goal, a simple yet effective sanitization instruction can be:

Do anything you are told to do in the following context.  
 Context: {Context}  
 Only do what the user asks you to do. Do not provide any explanation.  
 Your response:

**Characterizing instruction following behavior of the LLM:** Given a context and an LLM, we use the sanitization instruction to prompt the LLM to generate a single output token (denoted as  $o$ ) based on the given context to characterize the instruction following behavior of the LLM. Our insight for generating a single output token instead of the entire output is that: the attention weights between input tokens and the first generated output token already capture the dominant influence of the injected instruction. Thus,

analyzing this single-token generation significantly reduces computational cost while preserving sufficient information for effective sanitization.

**Sanitizing tokens that drive the instruction following behavior:** Given the output token  $o$  generated by an LLM using the sanitization instruction and a context  $C$ , we leverage the attention weights between each token  $c_k \in C$  and the output token  $o$  to determine whether the token should be sanitized from the context. In particular, we sanitize tokens in  $C$  that receive high attention weights from the generated output token  $o$ .

**Extract attention weights:** We extract attention weights between each input token  $c_k \in C$  in the context and the output token  $o$  across attention heads in all layers. For simplicity, we use  $\mathbf{A}_k \in \mathbb{R}^{L \times H}$  to denote the attention weights between the token  $c_k$  and the output token  $o$ , where  $L$  is the number of layers of the LLM and  $H$  is the number of attention heads. Each entry  $a_k^{ij} \in \mathbf{A}_k$  represents the attention weight between the token  $c_k \in C$  and the output token  $o$  in the  $j$ -th attention head at the  $i$ -th layer.

**Noise-aware aggregation of attention weights over attention heads:** Given  $\mathbf{A}_k$  for each token  $c_k \in C$ , a straightforward solution is to directly average the weights in  $\mathbf{A}_k$ , i.e.,  $s_k = \frac{1}{L \cdot H} \sum_{l=1}^L \sum_{h=1}^H a_k^{lh}$ . However, not all attention heads carry meaningful information. For instance, certain layers capture a strong influence of injected instructions on the output token, while other layers are less informative. If we directly average across all heads and layers, the less informative attention weights would dilute the important signals. In response, we propose a layer-wise noise-aware aggregation method. For each layer  $l \in [L]$ , we first compute the average attention over its heads:  $s_k^l = \frac{1}{H} \sum_{h=1}^H a_k^{lh}$ . Then, we take the maximum value across all layers:  $s_k = \max_{l \in [L]} s_k^l$ . We use  $\mathbf{s} = (s_1, s_2, \dots, s_m)$  to denote the vector of  $m$  aggregated attention weights for the  $m$  tokens in the context  $C$ . As shown in our results, this aggregation can effectively amplify salient attention weights associated with injected tokens while suppressing noise from less informative layers, thereby improving sanitization effectiveness.

**Jointly considering aggregated attention weights of consecutive tokens for sanitization:** Given the aggregated attention weight vector  $\mathbf{s}$ , one naive solution is to directly view each individual token  $c_k \in C$  whose aggregated attention weight  $s_k$  is larger than a threshold as injected one. However, this may achieve a sub-optimal performance. The major reason is that this does not jointly consider consecutive tokens and thus cannot capture the group effect of injected tokens.

To address the above limitation, we have two key insights. First, an injected instruction often consists of a sequence of tokens rather than isolated tokens. If one token is injected, its neighboring tokens are more likely to be injected. Second, when a segment of text contains an injected instruction, the consecutive tokens generally have consistently high attention weights due to their shared goal in inducing the LLM’s response. Based on these two insights, we perform following operations on  $\mathbf{s}$ . We first smooth  $\mathbf{s}$  to remove local noise and short-term fluctuations, allowing consecutive tokens with consistently high attention scores (corresponding to potential injected segments) to be identified more reliably. In particular, we use the Savitzky–Golay filter [53] for smoothing, which is a digital smoothing filter that preserves important features of the signal (e.g., peaks, widths, and relative maxima/minima) that is better than simple moving averages. To implement it, we use `scipy.signal.savgol_filter` from SciPy with a smooth window size  $w_s$  (a hyperparameter, e.g.,  $w_s = 5$ ). We use  $\bar{\mathbf{s}}$  to denote the smoothed  $\mathbf{s}$ .

Given the smoothed attention weight  $\bar{\mathbf{s}}$  for tokens in the context  $C$ , we find peaks in  $\bar{\mathbf{s}}$ , where tokens around each peak correspond to a continuous group of tokens with relatively high smoothed attention weights. We implemented this with `scipy.signal.find_peaks`, where we filter out peaks with very small smoothed attention weights. In general, each peak token and its surrounding tokens represent a short span of tokens that an LLM focuses on, often forming a meaningful phrase or instruction.

Given the identified peaks, we iteratively group nearby peaks together based on their distance. In particular, if the gap between two adjacent peaks is smaller than a predefined distance  $d$  (e.g., a hyperparameter, e.g.,  $d = 10$ ), we merge them into a single group. This step ensures that fragmented peaks originating from the same injected instruction are combined, yielding coherent segments that more accurately represent the complete injected instruction. We use  $P_1, P_2, \dots, P_e$  to denote  $e$  groups of peaks. For each  $P_i$ , we use  $G_i$

to denote a sequence of consecutive tokens surrounding the peaks in  $P_i$ . For each group  $G_i$ , we use  $v_i$  to denote the maximum aggregated attention weight for tokens in  $G_i$ , i.e.,  $v_i = \arg\max_{c_k \in G_i} s_k$ . Suppose  $i^*$  is the index whose  $v_i$  is the largest (we take the smallest index if a tie exists), i.e.,  $i^* = \arg\max_{i=1, \dots, e} v_i$ . We view the tokens in the group  $G_{i^*}$  as injected if  $v_{i^*}$  is larger than a threshold  $\theta$  (a hyperparameter); otherwise, they are regarded as clean. Note that  $\theta$  controls a utility-effectiveness tradeoff.

**Complete algorithm:** Algorithm 1 (in Appendix) shows the complete algorithm for PISanitizer.

An attacker may insert multiple injected instructions at different locations. Thus, we apply PISanitizer multiple times until no tokens are removed or a maximum number of repetitions (e.g., we set it to be 5 in experiments) is reached.

## 5 Evaluation

### 5.1 Experimental Setup

**Target tasks:** We use 6 datasets from the LongBench benchmark [54] to form our target tasks. LongBench [54] is a widely adopted benchmark designed to evaluate a long-context LLM’s performance on input prompts ranging from 4,000 to 20,000 tokens. It contains diverse applications such as question answering, summarization, code generation, and information retrieval, enabling a comprehensive evaluation of our PISanitizer under practical long-context scenarios. We use datasets from the following applications:

- **Question answering:** We use Qasper [55] and HotpotQA [56] for question answering. For Qasper, the task is to let an LLM answer a question based on an NLP research paper. HotpotQA is a multi-doc question answering task that requires an LLM to integrate information from multiple documents to answer questions.
- **Summarization:** GovReport [57] is a single-doc summarization task that require summarizing a government report, while MultiNews [58] is a multi-doc news summarization that requires summarizing over multiple news.
- **Code generation:** We use LCC [59] for code generation, where the task is to let an LLM generate the next line of code given a long piece of code.
- **Information retrieval:** PassageRetrieval is a dataset designed by LongBench [54], which simulates retrieval-based applications. Given 30 Wikipedia paragraphs and a summary, the task is to let an LLM determine which paragraph the given summary corresponds to.

Each dataset has 200 samples (or tasks), where each sample consists of a target instruction and a long context. For each dataset, we randomly select 100 samples for evaluation. Unless otherwise mentioned, we use HotpotQA as the default dataset for target tasks.

**Injected tasks:** We consider the following injected tasks: *target answer generation* [60] (the goal is to generate an attacker-chosen target answer), *hacked generation* [43] (the goal is to generate “Hacked!”), *classic NLP task* [4] (each injected task is a classic NLP task such as sentiment analysis), and *general instruction* [61] (contains diverse daily tasks). The details for these injected tasks are in Appendix A. Given the clean context for a target task, we can use a prompt injection attack to embed an injected task into the context. Unless otherwise mentioned, we use target answer generation as the default injected task.

**Prompt injection attacks:** We consider state-of-the-art *heuristic-based attacks* and *optimization-based attacks*.

- **Heuristic-based attacks:** We consider the following heuristic-based attacks: Naive Attack [2], Escape Characters [2], Context Ignoring [1], Fake Completion [10], and Combined Attack [4]. Among these attacks, Combined Attack achieves state-of-the-art performance as shown in [4]. We adopt the open-source implementation from [4].